

Introducción general

En este trabajo se desarrolla una exploración sobre el diseño y la optimización de prompts para un asistente de IA aplicado a la gestión institucional de IOMA.

El objetivo es mostrar cómo, a partir de un mismo caso de uso, la calidad, precisión y costo de las respuestas del modelo dependen directamente del nivel de detalle y de las técnicas empleadas en la construcción del prompt.

Para ello se parte de una **base de conocimiento real** (FAQs institucionales) y de una **misma consulta representativa**.

A lo largo del cuaderno se presentan distintas versiones de prompts, organizadas de manera progresiva:

- **Versión 1:** prompt mínimo, casi sin reglas.
- **Versión 2:** incorporación de rol, público y tono.
- **Versión 3:** primeras reglas y estructura básica.
- **Versión 4:** estructura más completa con subsecciones y límite de extensión, cerrada exclusivamente a la BASE.
- **Versión 5:** prompt mixto, donde el checklist se mantiene atado a la BASE, pero los términos clave y las buenas prácticas pueden enriquecerse con conocimiento externo.
- **Versión 6:** optimización orientada a **reducir costos**, aplicando técnicas de *Fast Prompting* (BASE compacta, reglas breves y salida en JSON conciso).

Cada versión se acompaña con su correspondiente explicación, ejecución práctica y cálculo de costos, lo que permite comparar la evolución en tres ejes:

1. **Calidad y claridad de la respuesta.**
2. **Grado de cumplimiento de las reglas definidas** (uso exclusivo de la BASE o apertura a conocimiento externo).
3. **Uso de tokens y costo asociado** en cada ejecución.

De este modo, se evidencia cómo el diseño de prompts es un proceso iterativo que combina creatividad, precisión y eficiencia, y cómo la aplicación de técnicas de *Fast Prompting* no solo mejora la confiabilidad de las respuestas, sino que también optimiza la **eficiencia económica** del sistema.

Preparación y funciones auxiliares

Antes de ejecutar los distintos prompts es necesario preparar la base de conocimiento y definir un conjunto de funciones que permitan seleccionar la FAQ más relevante según la consulta realizada.

Este bloque cumple tres objetivos principales:

1. **Carga y filtrado de la base**

- Se importa el archivo CSV con las FAQs.
- Se filtran únicamente las que tienen estado "vigente" o "en revisión".
- Se normalizan columnas clave para evitar valores nulos o inconsistencias.

2. Tokenización y relevancia

- `dividir_en_tokens(texto)` : transforma un texto en una lista de tokens en minúscula, eliminando signos de puntuación.
- `calcular_relevancia(tokens_consulta, fila)` : asigna un puntaje a cada fila de la base comparando los tokens de la consulta con las palabras clave, el título y el contenido. Se da más peso a las coincidencias en palabras clave ($\times 3$), luego en título ($\times 2$) y por último en contenido ($\times 1$).

3. Selección de FAQ

- `buscar_faq(consulta, tabla)` : utiliza las funciones anteriores para identificar qué fila de la base tiene mayor relevancia frente a la consulta ingresada.
- Si encuentra coincidencias, devuelve la FAQ más relevante; de lo contrario, indica que no hay resultados y que corresponde derivar el caso.

En síntesis, estas funciones permiten **conectar de forma eficiente la pregunta de un agente con la información normativa más pertinente**, asegurando que cada prompt se construya sobre la BASE adecuada.

```
In [28]: import pandas as pd
import re
from textwrap import dedent
import csv
from openai import OpenAI
from dotenv import load_dotenv
import os

# Cargar base de conocimiento
df = pd.read_csv("../data/base_conocimiento_afiliaciones_clean.csv", dtype=str,

# Filtrar por estado
df = df[df["estado"].str.lower().isin(["vigente", "en revisión"])]

# Normalizar columnas
for col in ["id", "titulo", "contenido", "respuesta_validada", "palabras_clave"]:
    if col in df.columns:
        df[col] = df[col].fillna("").astype(str)

# Funciones auxiliares
def dividir_en_tokens(texto: str):
    texto = texto.lower()
    return [t for t in re.split(r"^[a-zAÉÍÓÚÑÜ0-9]+", texto) if t]

def calcular_relevancia(tokens_consulta, fila):
    puntaje = 0
    puntaje += 3 * len(set(tokens_consulta) & set(dividir_en_tokens(fila.get("pa
    puntaje += 2 * len(set(tokens_consulta) & set(dividir_en_tokens(fila.get("ti
    puntaje += 1 * len(set(tokens_consulta) & set(dividir_en_tokens(fila.get("cc
    return puntaje
```

```
def buscar_faq(consulta: str, tabla: pd.DataFrame):
    toks_consulta = dividir_en_tokens(consulta)
    puntuadas = [(calcular_relevancia(toks_consulta, fila), idx) for idx, fila in enumerate(tabla.itertuples())]
    puntuadas = sorted(puntuadas, key=lambda x: x[0], reverse=True)
    if not puntuadas:
        return None
    puntuadas.sort(reverse=True)
    return tabla.loc[puntuadas[0][1]]
```

Configuración inicial

En este bloque se establece la configuración general necesaria para trabajar con la API de OpenAI.

Se cargan las variables de entorno con la clave de acceso, se inicializa el cliente y se define la consulta de prueba que utilizaremos a lo largo de todas las ejecuciones. Además, se selecciona de la base de conocimiento la FAQ más relevante frente a la consulta, sobre la cual se construirán los distintos prompts.

```
In [29]: # Configuración del cliente
load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

# Consulta de prueba
consulta = "¿Qué documentación necesito para afiliarse a un recién nacido?"

# Selección de fila relevante
fila_sel = buscar_faq(consulta, df)
if fila_sel is None:
    raise ValueError("No se encontró una FAQ relevante. Derivar a Afiliaciones (")

print("FAQ seleccionada:", fila_sel.get("id", "(sin id)"), "-", fila_sel.get("tit
```

FAQ seleccionada: faq_004 - Afiliación de recién nacido/a

Bloque 2 – Función genérica de ejecución

Aquí se define una función auxiliar `ejecutar_prompt`, que permite reutilizar el mismo flujo de implementación con cualquier versión del prompt.

La función recibe como parámetros:

- la función `construir_prompt_vX` que define la variante del prompt,
- la fila seleccionada de la base,
- la consulta planteada.

De este modo evitamos repetir código en cada prueba y podemos ejecutar fácilmente todas las versiones de prompt simplemente cambiando la función que se pasa como argumento.

Además, dentro de la misma función se incorporó el **cálculo automático de costos**, a partir de los tokens de entrada (prompt) y de salida (respuesta) que devuelve la API. Esto permite obtener en cada ejecución:

- Cantidad de tokens consumidos en el prompt.

- Cantidad de tokens generados en la respuesta.
- Total de tokens procesados.
- **Costo estimado en dólares** de esa llamada al modelo.

De esta manera, no solo se evalúa la calidad de las respuestas según cada versión de prompt, sino también su **eficiencia económica**, aspecto central en la optimización con *Fast Prompting*.

```
In [30]: # --- Precios por millón de tokens ---
PRECIOS_USD = {
    "gpt-4o-mini": {"in": 0.15, "out": 0.60}
}

def ejecutar_prompt(fn_prompt, fila_sel, consulta,
                   model="gpt-4o-mini", temperature=0.3, max_tokens=400):
    """
    fn_prompt: función construir_prompt_vX (ej. construir_prompt_v5)
    fila_sel:  fila seleccionada de la base
    consulta:  string con la pregunta del agente

    Muestra la respuesta del modelo + tokens consumidos y costo estimado (USD).
    """
    # 1) Construcción del prompt (system = reglas + BASE; user = consulta)
    prompt = fn_prompt(fila_sel, consulta)

    # 2) Llamada al modelo
    resp = client.chat.completions.create(
        model=model,
        messages=[
            {"role": "system", "content": prompt},
            {"role": "user", "content": consulta},
        ],
        temperature=temperature,
        max_tokens=max_tokens,
    )

    # 3) Mostrar respuesta
    respuesta = resp.choices[0].message.content
    print("\n--- RESPUESTA DEL MODELO ---\n")
    print(respuesta)

    # 4) Cálculo de costos
    precios = PRECIOS_USD.get(model, PRECIOS_USD["gpt-4o-mini"])
    pt = getattr(resp.usage, "prompt_tokens", 0)
    ct = getattr(resp.usage, "completion_tokens", 0)
    tt = getattr(resp.usage, "total_tokens", pt + ct)

    costo_in = (pt / 1_000_000) * precios["in"]
    costo_out = (ct / 1_000_000) * precios["out"]
    costo_tot = costo_in + costo_out

    print("\n--- COSTOS ---")
    print(f"Modelo: {model} | Temp: {temperature} | Máx. tokens salida: {max_tok}")
    print(f"Prompt tokens: {pt} | Completion tokens: {ct} | Total: {tt}")
    print(f"Costo entrada (USD): {costo_in:.8f}")
    print(f"Costo salida (USD): {costo_out:.8f}")
    print(f"Costo total (USD): {costo_tot:.8f}")
```

Versión 1 – Prompt básico (mínimo)

Comenzamos con un prompt muy simple, casi sin reglas.

El asistente solo recibe la pregunta y la base como contexto, y se le pide responder.

No hay estructura definida ni control sobre la extensión o la fidelidad de la respuesta.

Esta versión nos sirve como **punto de partida** para observar cómo responde el modelo con indicaciones mínimas.

```
In [31]: # VERSIÓN 1 - MUY BÁSICO (casi sin reglas)
def construir_prompt_v1(fila, pregunta: str):
    base = (fila.get("contenido") or fila.get("titulo", "")).strip()
    return f"""
    Pregunta: {pregunta}
    Base: {base}
    Responde usando la base.
    """

ejecutar_prompt(construir_prompt_v1, fila_sel, consulta)
```

--- RESPUESTA DEL MODELO ---

Para afiliar a un recién nacido, necesitarás presentar la documentación correspondiente en la Delegación de IOMA que corresponda al domicilio de la persona afiliada titular, o puedes realizar el trámite a través de la App IOMA Digital/Autogestión. Asegúrate de tener a mano el documento de identidad del recién nacido y cualquier otro requisito que pueda ser solicitado en el momento del trámite.

--- COSTOS ---

Modelo: gpt-4o-mini | Temp: 0.3 | Máx. tokens salida: 400

Prompt tokens: 86 | Completion tokens: 81 | Total: 167

Costo entrada (USD): 0.00001290

Costo salida (USD): 0.00004860

Costo total (USD): 0.00006150

Versión 2 – Rol y tono

En esta segunda versión agregamos información contextual:

- Definimos un **rol** ("Asistente de Afiliaciones de IOMA").
- Establecemos el **público** (agentes administrativos).
- Indicamos un **tono** (claro, formal e institucional).

Esto corresponde a una de las técnicas de *Fast Prompting*: **dar contexto explícito al modelo**.

Al asignar un rol, un público objetivo y un tono, se reduce la ambigüedad y se orienta la respuesta hacia el estilo esperado, evitando desvíos en la redacción.

De esta manera empezamos a darle al modelo un marco más controlado, aunque todavía sin imponer una estructura rígida en la salida.

```
In [32]: # VERSIÓN 2 - CON ROL Y TONO
def construir_prompt_v2(fila, pregunta: str):
```

```

base = (fila.get("contenido") or fila.get("titulo","")).strip()
return f"""
Rol: Asistente de Afiliaciones de IOMA
Público: agentes administrativos
Tono: claro, formal, institucional

Pregunta: {pregunta}

Base de referencia:
{base}

Responde de forma clara y breve usando SOLO la base.
"""
ejecutar_prompt(construir_prompt_v2, fila_sel, consulta)

```

--- RESPUESTA DEL MODELO ---

Para afiliar a un recién nacido, se debe presentar la documentación correspondiente en la Delegación de IOMA correspondiente al domicilio del afiliado titular, o a través de la App IOMA Digital/Autogestión.

--- COSTOS ---

Modelo: gpt-4o-mini | Temp: 0.3 | Máx. tokens salida: 400
 Prompt tokens: 124 | Completion tokens: 44 | Total: 168
 Costo entrada (USD): 0.00001860
 Costo salida (USD): 0.00002640
 Costo total (USD): 0.00004500

Versión 3 – Estructura básica y reglas

Aquí damos un salto importante:

- Se aclara que el modelo debe usar **exclusivamente la BASE** como fuente de información.
- Se introduce la regla de que, si falta un dato, debe indicarlo explícitamente con un mensaje predefinido.
- Se exige una **estructura mínima de salida**, que incluye un checklist de pasos y un cierre obligatorio con la frase exacta.

En esta versión se aplican técnicas de *Fast Prompting* más avanzadas que en la anterior:

- **Grounding estricto en la BASE**, para reducir invención o “alucinación” del modelo.
- **Instrucciones de fallback** (qué hacer si falta información), lo que garantiza consistencia en todos los casos.
- **Estructuración de la salida**, que disminuye la ambigüedad y facilita la validación posterior.

De este modo, el prompt pasa de ser meramente contextual (Versión 2) a estar controlado por **reglas claras y un formato mínimo esperado**, aumentando la confiabilidad de la respuesta.

```

In [33]: # VERSIÓN 3 - CON ESTRUCTURA Y REGLAS
def construir_prompt_v3(fila, pregunta: str):
    base = (fila.get("contenido") or fila.get("titulo","")).strip()
    return f"""

```

Rol: Asistente de Afiliaciones de IOMA
Público: agentes en Delegaciones
Tono: institucional, claro y preciso

Reglas:

- Basate EXCLUSIVAMENTE en la Base.
- Si falta un dato, decí "No consta en la normativa adjunta".

Respuesta debe incluir:

1. Pasos o checklist (en viñetas).
2. Cierre con: "Fuente: base de conocimiento vigente".

Pregunta: {pregunta}

BASE:

{base}
""

```
ejecutar_prompt(construir_prompt_v3, fila_sel, consulta)
```

--- RESPUESTA DEL MODELO ---

Para afiliar a un recién nacido, se deben seguir los siguientes pasos:

- Iniciar el trámite en la Delegación correspondiente al domicilio de la persona afiliada titular, o a través de la App IOMA Digital/Autogestión.
- Presentar la documentación necesaria (no consta en la normativa adjunta).

Por favor, asegúrese de tener a mano toda la información requerida para facilitar el proceso.

Fuente: base de conocimiento vigente.

--- COSTOS ---

Modelo: gpt-4o-mini | Temp: 0.3 | Máx. tokens salida: 400

Prompt tokens: 181 | Completion tokens: 90 | Total: 271

Costo entrada (USD): 0.00002715

Costo salida (USD): 0.00005400

Costo total (USD): 0.00008115

Versión 4 – Prompt más detallado con subsecciones (cerrado a la BASE)

En esta versión el prompt se vuelve más completo y estructurado:

- Se fija un **límite máximo de extensión** (≤ 350 palabras).
- Se exige un **cierre exacto** con la frase "Fuente: base de conocimiento vigente".
- Se introduce un **contexto ampliado** con dos subsecciones:
 - **Términos clave** (≥ 3 ítems).
 - **Objetivo y buenas prácticas** (≥ 3 ítems).

La técnica de *Fast Prompting* aplicada aquí es el **prompt estructurado con subsecciones y restricciones de formato**.

Esto aporta varios beneficios:

- **Control de longitud** → evitar respuestas excesivas y optimizar el uso de tokens.

- **Formato obligatorio con subsecciones** → fuerza al modelo a organizar la salida en bloques predefinidos.
- **Grounding cerrado en la BASE** → asegura que todo el contenido provenga exclusivamente de la normativa institucional, sin invenciones externas.

De esta manera, se logra un alto grado de control sobre la respuesta, a costa de un mayor consumo de tokens.

```
In [34]: # VERSIÓN 4 - MÁS DETALLADA (subsecciones y límites)
def construir_prompt_v4(fila, pregunta: str):
    base = (fila.get("respuesta_validada") or
            fila.get("contenido") or
            fila.get("titulo", "")).strip()
    return f"""
    Rol: Asistente de Afiliaciones de IOMA
    Público: agentes en Delegaciones
    Tono: institucional, claro y preciso

    Reglas:
    - Basate EXCLUSIVAMENTE en la BASE que está entre <<BASE>> y <<FIN_BASE>>.
    - Si un dato, definición o recomendación NO aparece en la BASE, escribí lite
    - Está prohibido inventar o usar información externa.
    - Extensión total ≤ 350 palabras.
    - El cierre debe ser EXACTAMENTE: "Fuente: base de conocimiento vigente".

    La respuesta debe incluir:
    1. Un único bloque titulado **Checklist de documentación requerida** con la
       No repitas la misma lista fuera de esta sección.
    2. Cierre exacto con la frase indicada.
    3. Contexto ampliado con dos subsecciones:
       - Términos clave: ≥3 ítems, cada uno en el formato "Término: definición".
       - Objetivo y buenas prácticas: ≥3 ítems, cada uno en el formato "Práctica

    Pregunta: "{pregunta}"

    <<BASE>>
    {base}
    <<FIN_BASE>>
    """.strip()

ejecutar_prompt(construir_prompt_v4, fila_sel, consulta)
```


--- RESPUESTA DEL MODELO ---

****Checklist de documentación requerida****

1. Credencial o DNI del afiliado/a titular.
2. DNI del menor o constancia de parto donde figuren madre y padre.
3. Ficha de afiliación cumplimentada (no certificada, solo la parte del afiliado).
4. Último recibo de haberes.
5. CUIL si lo posee.
6. Certificado de nacimiento del menor.
7. Si la afiliación se realiza sin el DNI del niño/a, deberá presentarse obligatoriamente una vez obtenido.

****Términos clave:****

- ****Afiliado/a titular:**** Persona que tiene la responsabilidad de incluir a su hijo/a en el sistema de salud.
- ****Constancia de parto:**** Documento que acredita el nacimiento del menor y debe incluir los datos de los padres.
- ****Ficha de afiliación:**** Formulario que se debe completar para gestionar la inclusión del recién nacido en el sistema.

****Objetivo y buenas prácticas:****

- ****Objetivo de la afiliación:**** Garantizar el acceso a la salud del recién nacido a través de la inclusión en el sistema de IOMA.
- ****Presentación de documentación completa:**** Asegurarse de que toda la documentación requerida esté completa y correcta para evitar demoras en el proceso de afiliación.
- ****Seguimiento del DNI del menor:**** Si se inicia el proceso de afiliación sin el DNI del niño/a, es fundamental realizar un seguimiento para presentar este documento tan pronto como esté disponible.

Fuente: base de conocimiento vigente.

--- COSTOS ---

Modelo: gpt-4o-mini | Temp: 0.3 | Máx. tokens salida: 400

Prompt tokens: 422 | Completion tokens: 308 | Total: 730

Costo entrada (USD): 0.00006330

Costo salida (USD): 0.00018480

Costo total (USD): 0.00024810

Versión 5 – Prompt mixto (BASE + conocimiento externo)

En esta versión se separa explícitamente el **origen de la información** por sección:

- **Checklist de documentación** → se extrae **solo** de la BASE (normativa) para garantizar exactitud y trazabilidad.
- **Términos clave y Objetivo y buenas prácticas** → el modelo puede usar **conocimiento externo** para enriquecer la explicación con definiciones y recomendaciones claras, aun cuando la normativa no las explicita.

Técnicas de *Fast Prompting* aplicadas:

- **Delimitadores de contexto** (<<BASE>> ... <<FIN_BASE>>) → aíslan el texto normativo y reducen la ambigüedad.

- **Ámbito por sección (scoped prompting)** → define qué partes deben basarse en la BASE y cuáles pueden abrirse al conocimiento externo, evitando confusiones.
- **Estructura de salida en JSON** → asegura respuestas consistentes, fáciles de validar y de reutilizar en sistemas posteriores.

Este diseño introduce un equilibrio novedoso: mantiene el rigor normativo en los apartados críticos, pero aprovecha el conocimiento del modelo para agregar valor pedagógico en las explicaciones, mostrando cómo *Fast Prompting* permite modular el alcance de la respuesta según la sección.

```
In [35]: def construir_prompt_v5(fila, pregunta: str):
    base = (fila.get("respuesta_validada") or
            fila.get("contenido") or
            fila.get("titulo", "")).strip()
    return f"""
    Rol: Asistente de Afiliaciones de IOMA
    Público: agentes en Delegaciones
    Tono: institucional y claro

    Instrucciones por sección (ámbito de información):
    - "checklist": basate EXCLUSIVAMENTE en lo que está entre <<BASE>> y <<FIN_B
      Si falta un dato, escribir: "No consta en la normativa adjunta".
    - "terminos_clave" y "objetivo_y_buenas_practicas": podés usar tu conocimien
      Incluir definiciones y recomendaciones breves, claras y relevantes. Si no
      información externa, escribir "No consta".

    Formato de salida (JSON válido, sin texto adicional):
    {{
      "checklist": ["Documento 1", "Documento 2", "..."],
      "terminos_clave": ["Término: definición breve", "Término: definición breve
      "objetivo_y_buenas_practicas": ["Buena práctica: detalle breve", "Buena pr
      "cierre": "Fuente: base de conocimiento vigente"
    }}

    Reglas generales:
    - No inventes información normativa en el checklist: debe provenir de <<BASE
    - El JSON debe ser la única salida (no agregues texto fuera del objeto).
    - Mantener redacción clara y precisa.

    Pregunta: "{pregunta}"

    <<BASE>>
    {base}
    <<FIN_BASE>>
    """.strip()
    ejecutar_prompt(construir_prompt_v5, fila_sel, consulta)
```

--- RESPUESTA DEL MODELO ---

```
{
  "checklist": [
    "Credencial o DNI del afiliado/a titular",
    "DNI del menor o constancia de parto donde figuren madre y padre",
    "Ficha de afiliación cumplimentada (no certificada, solo la parte del afiliado)",
    "Último recibo de haberes",
    "CUIL si lo posee",
    "Certificado de nacimiento del menor"
  ],
  "terminos_clave": [
    "Afiliación: proceso mediante el cual se incorpora a una persona a un sistema de salud o seguridad social.",
    "DNI: Documento Nacional de Identidad, identificación oficial en Argentina.",
    "CUIL: Código Único de Identificación Laboral, número que identifica a los trabajadores en el sistema de seguridad social."
  ],
  "objetivo_y_buenas_practicas": [
    "Verificar que toda la documentación esté completa antes de iniciar el proceso de afiliación.",
    "Asegurarse de que la ficha de afiliación esté correctamente cumplimentada.",
    "Informar al afiliado/a titular sobre la importancia de presentar el DNI del menor una vez obtenido."
  ],
  "cierre": "Fuente: base de conocimiento vigente"
}
```

--- COSTOS ---

Modelo: gpt-4o-mini | Temp: 0.3 | Máx. tokens salida: 400
Prompt tokens: 482 | Completion tokens: 253 | Total: 735
Costo entrada (USD): 0.00007230
Costo salida (USD): 0.00015180
Costo total (USD): 0.00022410

Versión 6 – Prompt optimizado en costos (Fast Prompting)

En esta última versión, el objetivo principal es **optimizar el uso de tokens** para reducir los costos de cada llamada al modelo, manteniendo el control sobre la estructura de salida.

Características principales:

- **Prompt más breve (lean prompting):** reglas condensadas y redactadas con frases cortas.
- **BASE compacta:** antes de generar el prompt, la función auxiliar `compactar_texto` recorta y limpia la BASE a un máximo de caracteres, eliminando redundancias y evitando tokens innecesarios.
- **Formato en JSON conciso:** salida estructurada, fácil de validar y con menor extensión que una redacción en prosa.
- **Ámbito mixto:**
 - *Checklist* → restringido a la BASE (máxima confiabilidad normativa).
 - *Términos clave y Buenas prácticas* → permitidos con conocimiento externo (para enriquecer la explicación).

Técnicas de *Fast Prompting* aplicadas:

- **Delimitadores** (`<<BASE>>` ... `<<FIN_BASE>>`) → aíslan con claridad el contenido normativo.
- **Output en JSON** → controla la extensión y facilita la validación automática.
- **Rol + reglas explícitas pero compactas** → reducen tokens innecesarios sin perder claridad.
- **Self-contained instruction** → cada instrucción está contenida en el mismo bloque, evitando dependencias externas y asegurando consistencia.
- **Compactación de la BASE** (`compactar_texto`) → ejemplo concreto de optimización de entrada, clave para reducir costos.

Este diseño demuestra cómo el *Fast Prompting* no solo impacta en la **calidad de la respuesta**, sino también en la **eficiencia económica** del sistema.

En las pruebas, la Versión 6 mantuvo la riqueza informativa de la Versión 5 pero con menor consumo de tokens, alcanzando un costo promedio cercano a **0.00020 USD por consulta**. De esta forma, se consigue un equilibrio entre **precisión normativa, claridad pedagógica y optimización de recursos** en escenarios de producción.

```
In [ ]: def compactar_texto(s: str, max_chars=800):
        """Compacta espacios y recorta la BASE a un máximo de caracteres."""
        s = re.sub(r"\s+", " ", s).strip()
        return s[:max_chars]

def construir_prompt_v6(fila, pregunta: str):
    base = (fila.get("respuesta_validada") or
            fila.get("contenido") or
            fila.get("titulo", "")).strip()
    base = compactar_texto(base, 800) # compactar para ahorrar tokens

    return f"""
    Rol: Asistente de Afiliaciones de IOMA. Público: agentes. Tono: institucional

    Instrucciones:
    - "checklist": SOLO de <<BASE>>. Si falta dato: "No consta en la normativa a
    - "terminos_clave" y "objetivo_y_buenas_practicas": podés usar conocimiento
    - Responder SOLO con JSON válido.

    Formato:
    {{
      "checklist": ["...", "..."],
      "terminos_clave": ["Término: definición breve", "...", "..."],
      "objetivo_y_buenas_practicas": ["Buena práctica: detalle breve", "...", "..."],
      "cierre": "Fuente: base de conocimiento vigente"
    }}

    Pregunta: "{pregunta}"

    <<BASE>>
    {base}
    <<FIN_BASE>>
    """.strip()
_ = ejecutar_prompt(
    construir_prompt_v6,
    fila_sel,
```

```
consulta,  
model="gpt-4o-mini",  
temperature=0.1,  
max_tokens=250  
)
```

--- RESPUESTA DEL MODELO ---

```
{  
  "checklist": [  
    "Credencial o DNI del afiliado/a titular",  
    "DNI del menor o constancia de parto donde figuren madre y padre",  
    "Ficha de afiliación cumplimentada (no certificada, solo la parte del afiliado)",  
    "Último recibo de haberes",  
    "CUIL si lo posee",  
    "Certificado de nacimiento del menor"  
  ],  
  "terminos_clave": [  
    "DNI: Documento Nacional de Identidad, identificación oficial en Argentina.",  
    "CUIL: Código Único de Identificación Laboral, número que identifica a los trabajadores en el sistema de seguridad social.",  
    "Constancia de parto: Documento que acredita el nacimiento de un menor, indicando los datos de los padres."  
  ],  
  "objetivo_y_buenas_practicas": [  
    "Asegurar la correcta afiliación del recién nacido para garantizar el acceso a la salud.",  
    "Mantener la documentación actualizada y completa para evitar inconvenientes en el proceso de afiliación.",  
    "Realizar la afiliación lo antes posible tras el nacimiento para asegurar la cobertura desde el inicio."  
  ],  
  "cierre": "Fuente: base de
```

--- COSTOS ---

Modelo: gpt-4o-mini | Temp: 0.1 | Máx. tokens salida: 250
Prompt tokens: 336 | Completion tokens: 250 | Total: 586
Costo entrada (USD): 0.00005040
Costo salida (USD): 0.00015000
Costo total (USD): 0.00020040

Conclusiones finales

A lo largo de las seis versiones se observa una clara evolución tanto en la **calidad estructural de las respuestas** como en la **eficiencia económica**.

La siguiente tabla resume el costo por consulta y la proyección semanal para 100 consultas:

Versión	Prompt tokens	Completion tokens	Total tokens	Costo por consulta (USD)	Costo semanal (100 consultas) (USD)
V1	86	66	152	0.00005250	0.00525
V2	124	42	166	0.00004380	0.00438
V3	181	85	266	0.00007815	0.00782

Versión	Prompt tokens	Completion tokens	Total tokens	Costo por consulta (USD)	Costo semanal (100 consultas) (USD)
V4	427	342	769	0.00026925	0.02693
V5	482	252	734	0.00022350	0.02235
V6	336	250	586	0.00020040	0.02004

Análisis:

- **Versión 1 y 2:** son las más simples y de menor costo, pero con respuestas poco estructuradas y menos útiles en un entorno real.
- **Versión 3:** introduce reglas básicas y estructura, lo que mejora la claridad, aunque incrementa el costo.
- **Versión 4:** aporta mayor control con subsecciones y extensión limitada, pero al estar cerrada exclusivamente a la BASE genera la respuesta más costosa (≈ 0.027 USD/semana por 100 consultas).
- **Versión 5:** incorpora el modelo mixto (BASE para checklist y externo para definiciones/buenas prácticas). Se logra una respuesta más pedagógica, con costo menor que V4 pero aún alto respecto a las primeras.
- **Versión 6:** aplica técnicas de *Fast Prompting* (BASE compacta, reglas breves, JSON conciso). Mantiene la riqueza de V5 pero optimiza tokens, reduciendo el costo semanal a **≈ 0.020 USD por 100 consultas**.

Balance general:

El análisis muestra que un diseño de prompt más estructurado implica **más tokens y mayor costo**, pero que con optimizaciones de *Fast Prompting* (V6) es posible alcanzar un equilibrio: respuestas claras, normativamente correctas y con bajo costo operativo.